

Pre-Training: Auto-Parallelization

Adam Kuca, Qilong Wang, Yung-Hao Liao, Zhe Jiang

1/29/2026

Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning

Background: Distributed Deep Learning

Modern LLMs are getting larger for better performance

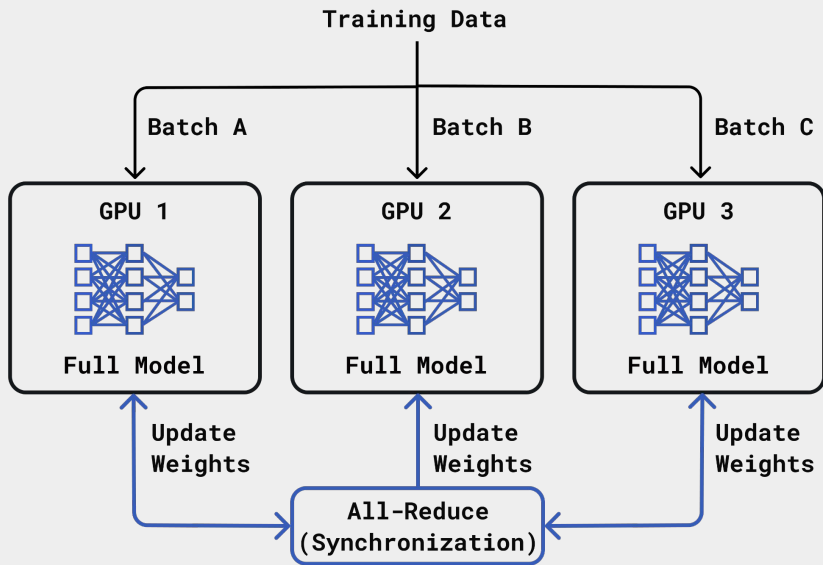
- Data
- Model

Q: Can't fit in a single machine. How to solve this?

→ Parallelism



Data Parallelism

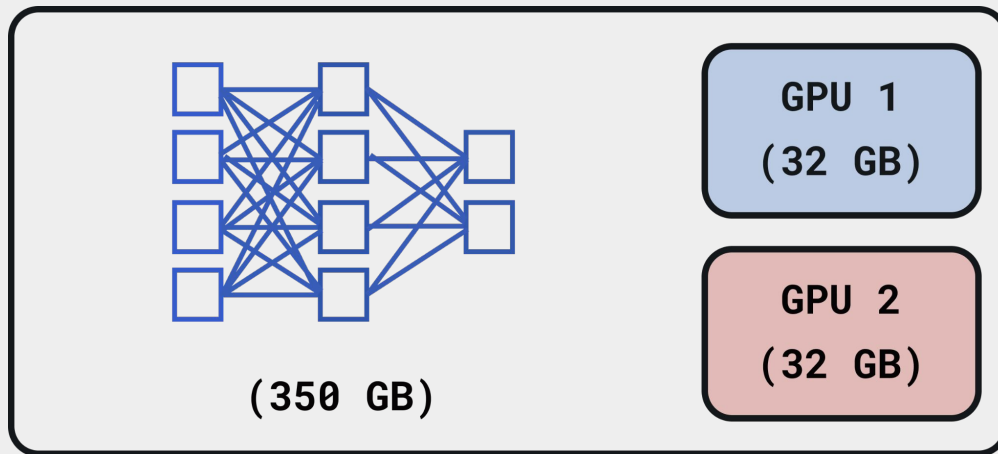


1. Replicate Model
2. Split Data
3. Synchronize Gradients

E.g. Pytorch DDP

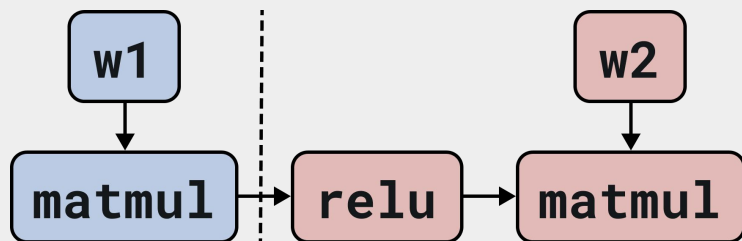
MODEL MUST FIT INTO MEMORY!

What if the model is too large?

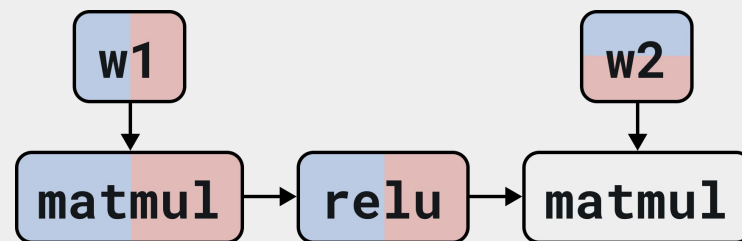


How to partition the computational graph?

Operator Parallelism

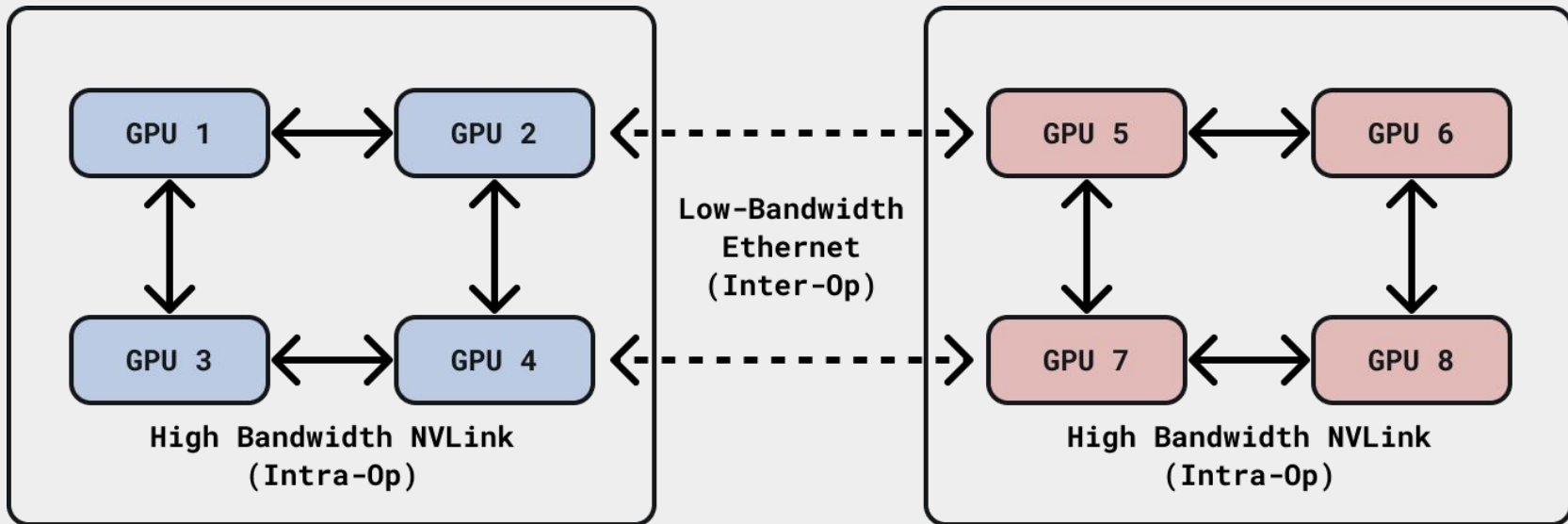


Inter-Operator Parallelism



Intra-Operator Parallelism

Why Inter- and Intra-Operator?

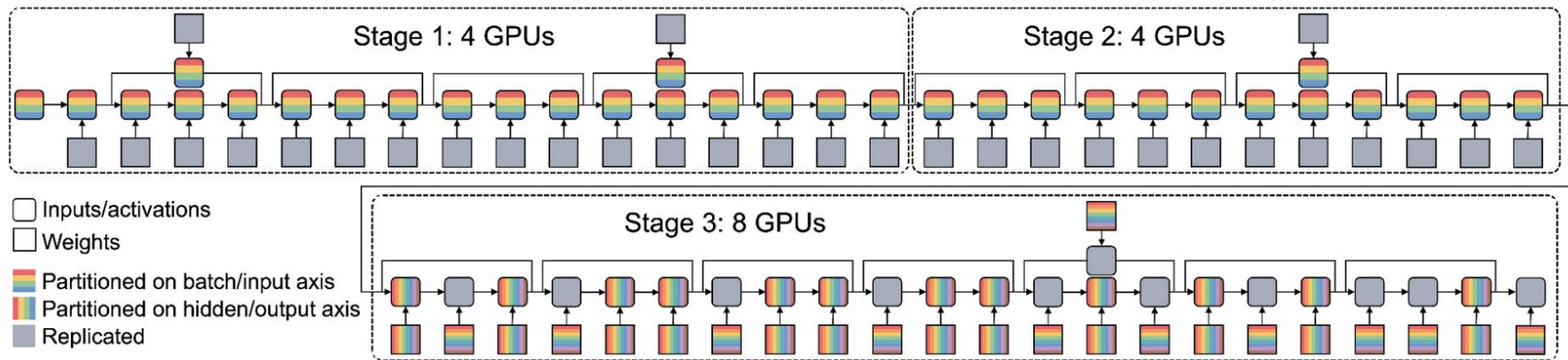




Operator Parallelism

1. **Inter-operator parallelism** (e.g. device placement, pipeline parallelism and their variants)
 - different operators to different devices
 - device underutilization (“bubbles”) - dependency issues
2. **Intra-operator parallelism** (e.g. data parallelism, Megatron-LM’s tensor model parallelism)
 - partition operators and execute different partitions in parallel
 - heavier communication - high bandwidth required

Problem with Manual Annotation



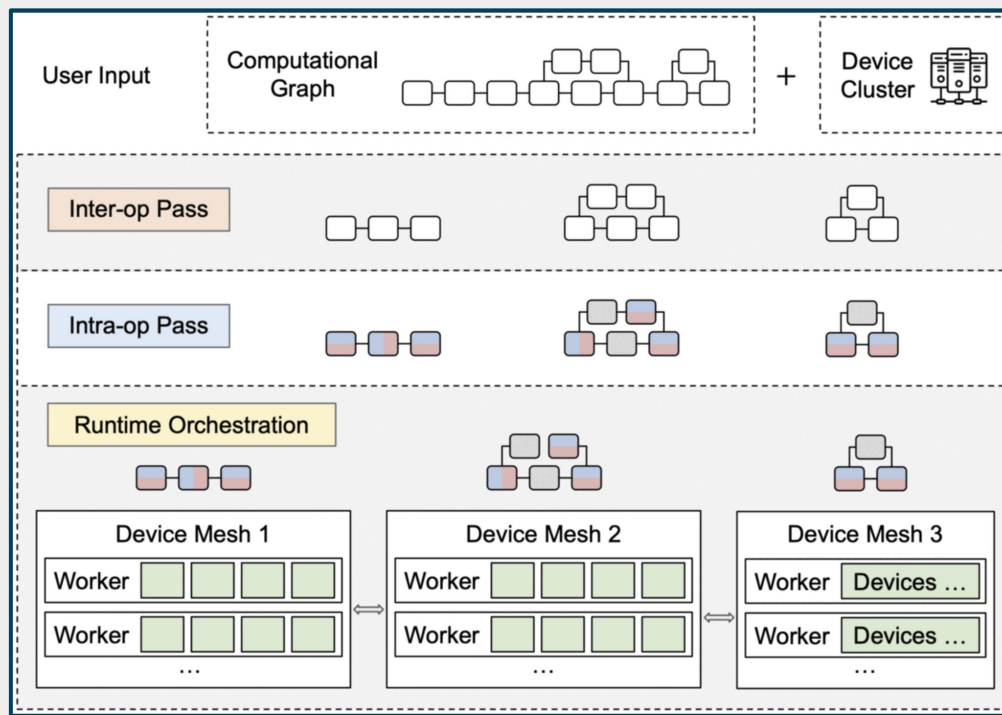
The parallelization strategy for Wide-ResNet on 16 GPUs consists of three pipeline stages and is a complicated strategy even for an expert to design. Stages 1 and 2 are on 4 GPUs performing data parallelism, and stage 3 is on 8 GPUs performing operator parallelism.



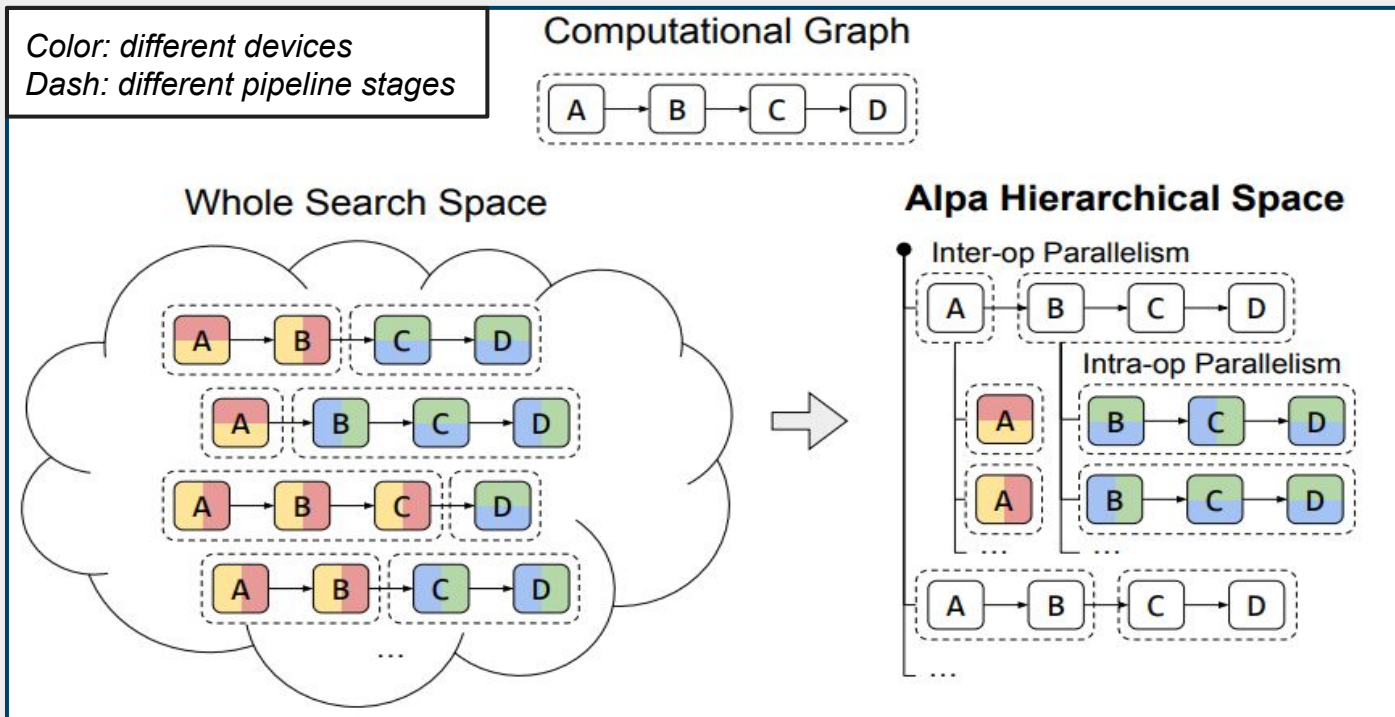
What is Alpa?

*“Alpa is a **compiler** which **automates** model-parallel training of large deep learning (DL) models by generating execution plans that unify data, operator, and pipeline parallelism.”*

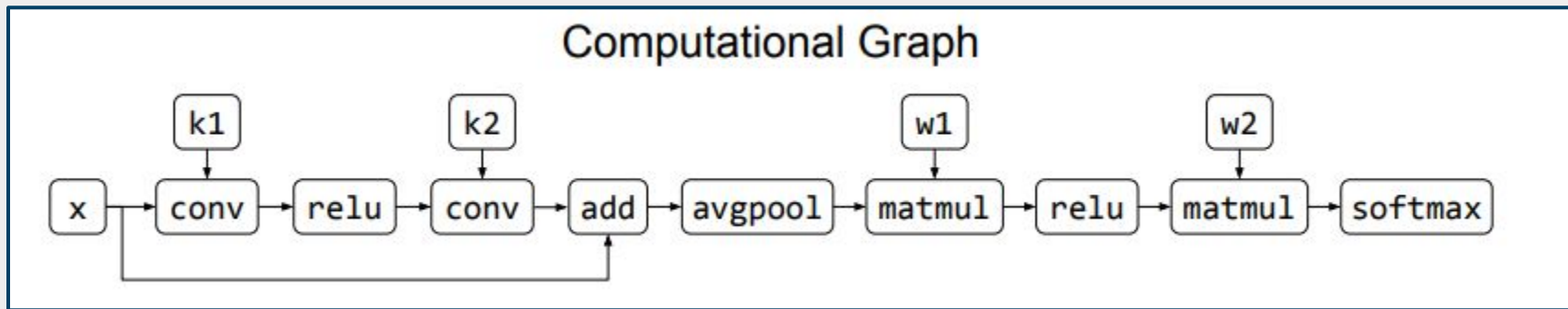
Structure of Alpa



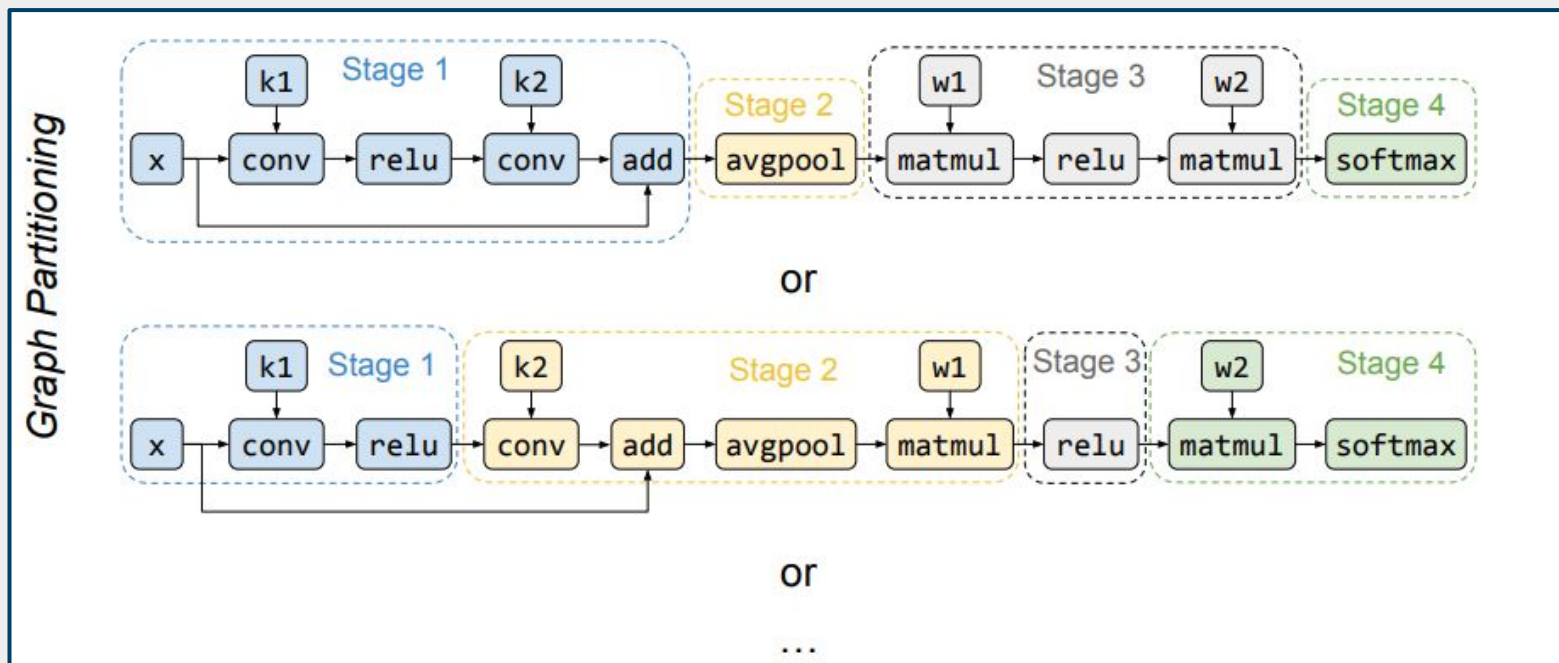
What is Alpa?: Search method



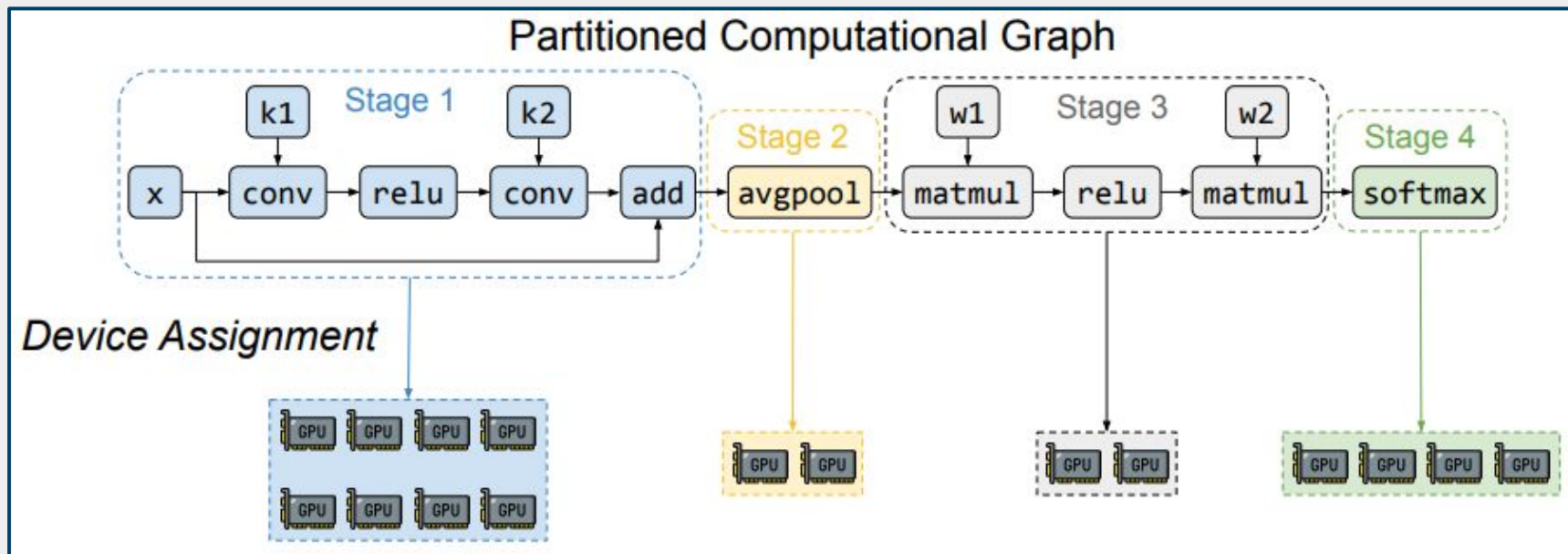
Inter-Operator Parallelism (Pass 1)



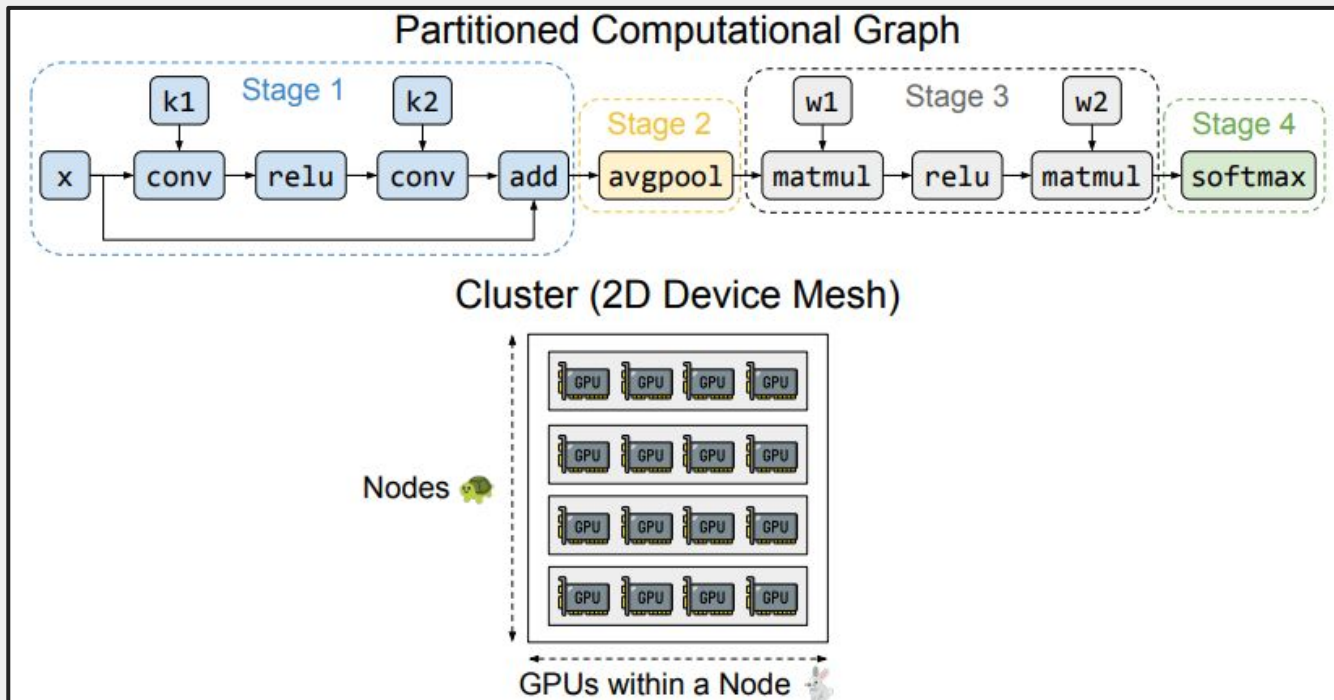
Inter-Operator Parallelism (Pass 1)



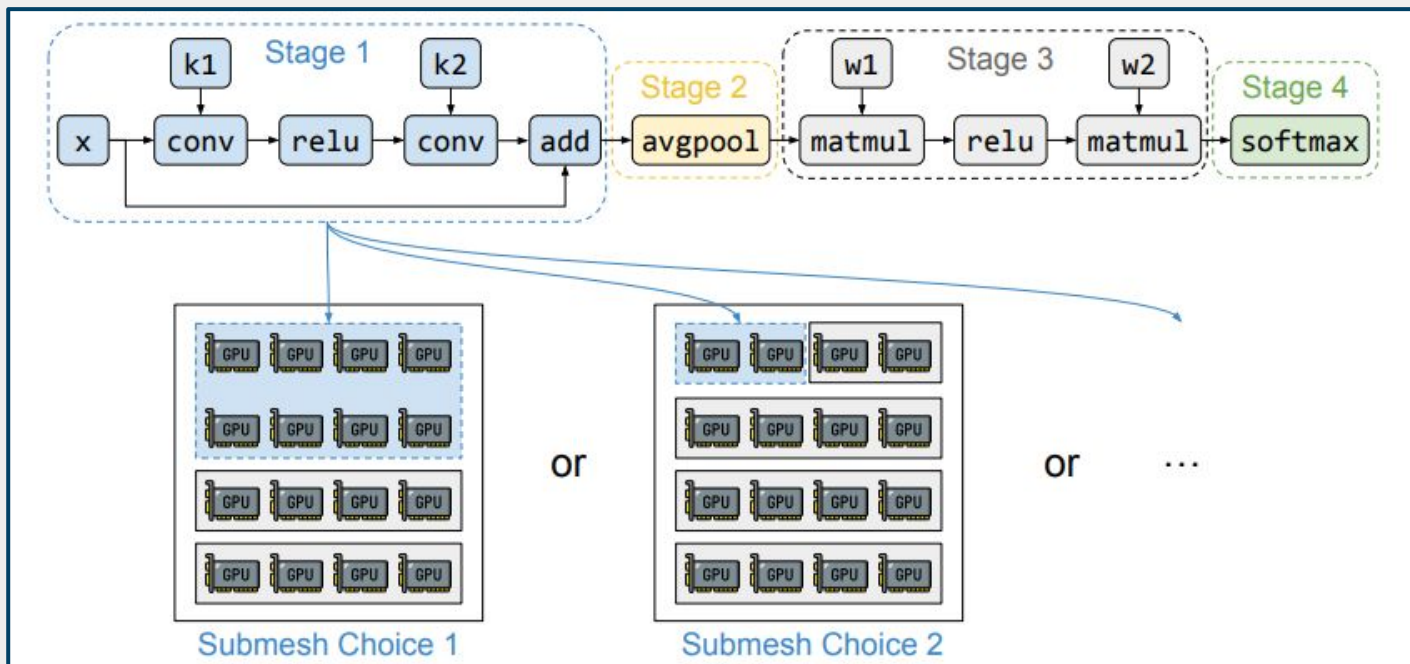
Inter-Operator Parallelism (Pass 1)



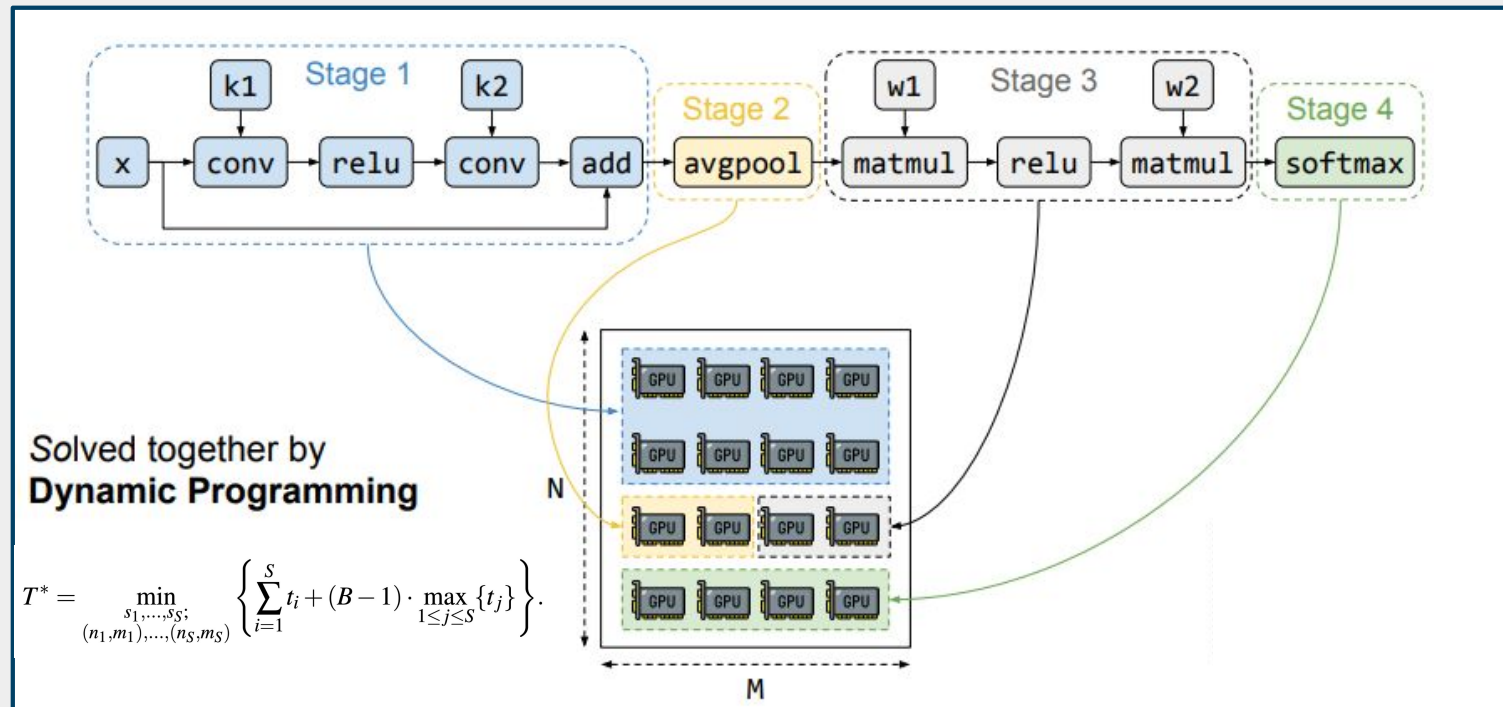
Inter-Operator Parallelism (Pass 1)



Inter-Operator Parallelism (Pass 1)



Inter-Operator Parallelism (Pass 1)



Inter-Operator Parallelism (Pass 1)

$$T^* = \min_{\substack{s_1, \dots, s_S; \\ (n_1, m_1), \dots, (n_S, m_S)}} \left\{ \underbrace{\sum_{i=1}^S t_i}_{(1)} + (B-1) \cdot \underbrace{\max_{1 \leq j \leq S} \{t_j\}}_{(2)} \right\}.$$

(1) Total pipeline latency time of all stages

(2) Pipelined execution time for the rest of B-1 microbatches

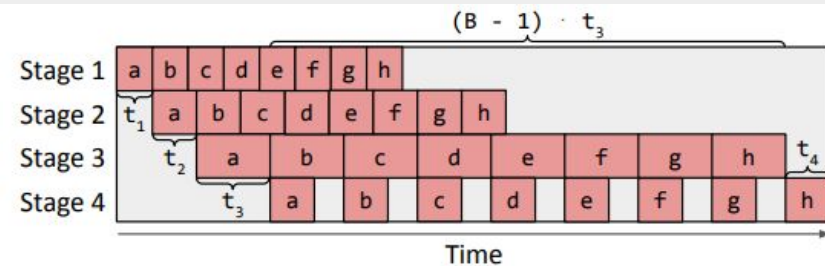


Figure 5: Illustration of the total latency of a pipeline, which is determined by two parts: the total latency of all stages ($t_1 + t_2 + t_3 + t_4$) and the latency of the slowest stage ($(B-1) \cdot t_3$).

Inter-Operator Parallelism (Pass 1)

$$F(s, k, d; t_{max})$$

(1)

Pipeline slicing

$$= \min_{\substack{k \leq i \leq K \\ n_s \cdot m_s \leq d}} \left\{ \begin{array}{l} t_{intra}((o_k, \dots, o_i), \text{Mesh}(n_s, m_s), s) \\ + F(s-1, i+1, d - n_s \cdot m_s; t_{max}) \\ | t_{intra}((o_k, \dots, o_i), \text{Mesh}(n_s, m_s), s) \leq t_{max} \end{array} \right\},$$

(2)

(3)

(1) How many latency the remaining stages require

- s (stages): How many stages left
- k: current operator
- d (devices): How many devices left
- t_{max} : bottleneck constraint

(2) Current Stage execution time

+ Remaining time for all stages' execution (Recursive)

(3) Constraint, abandon if $> t_{max}$

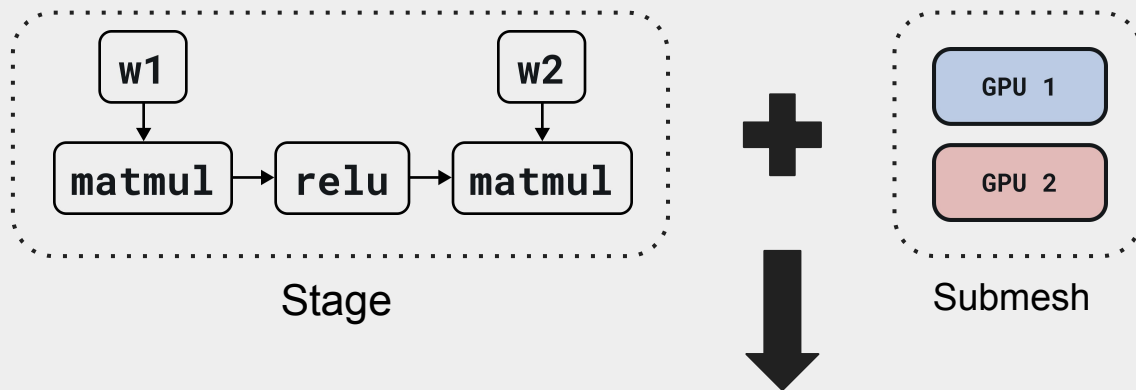
Inter-Operator Parallelism (Pass 1)

$$T^* = \min_{\substack{s_1, \dots, s_S; \\ (n_1, m_1), \dots, (n_S, m_S)}} \left\{ \sum_{i=1}^S t_i + (B-1) \cdot \max_{1 \leq j \leq S} \{t_j\} \right\}.$$



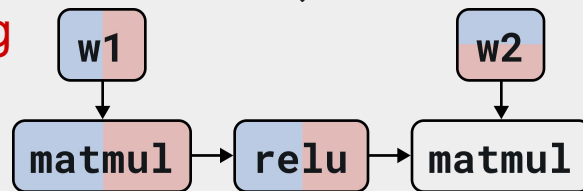
$$T^*(t_{max}) = \min_s \{ F(s, 0, N \cdot M; t_{max}) \} + (B-1) \cdot t_{max}.$$

Intra-Operator Parallelism (Pass 2)



Solved by Integer Linear Programming

$$\min_s \sum_{v \in V} s_v^T (c_v + d_v) + \sum_{(v,u) \in E} s_v^T R_{vu} s_u,$$



Intra-Operator Parallelism (Pass 2)

Integer Linear Programming Formulation

Computation + Communication Cost of node v

Resharding cost of edge (v,u)

$$\min_s \sum_{v \in V} s_v^T (c_v + d_v) + \sum_{(v,u) \in E} s_v^T R_{vu} s_u,$$

Focus on com cost

Decision vector

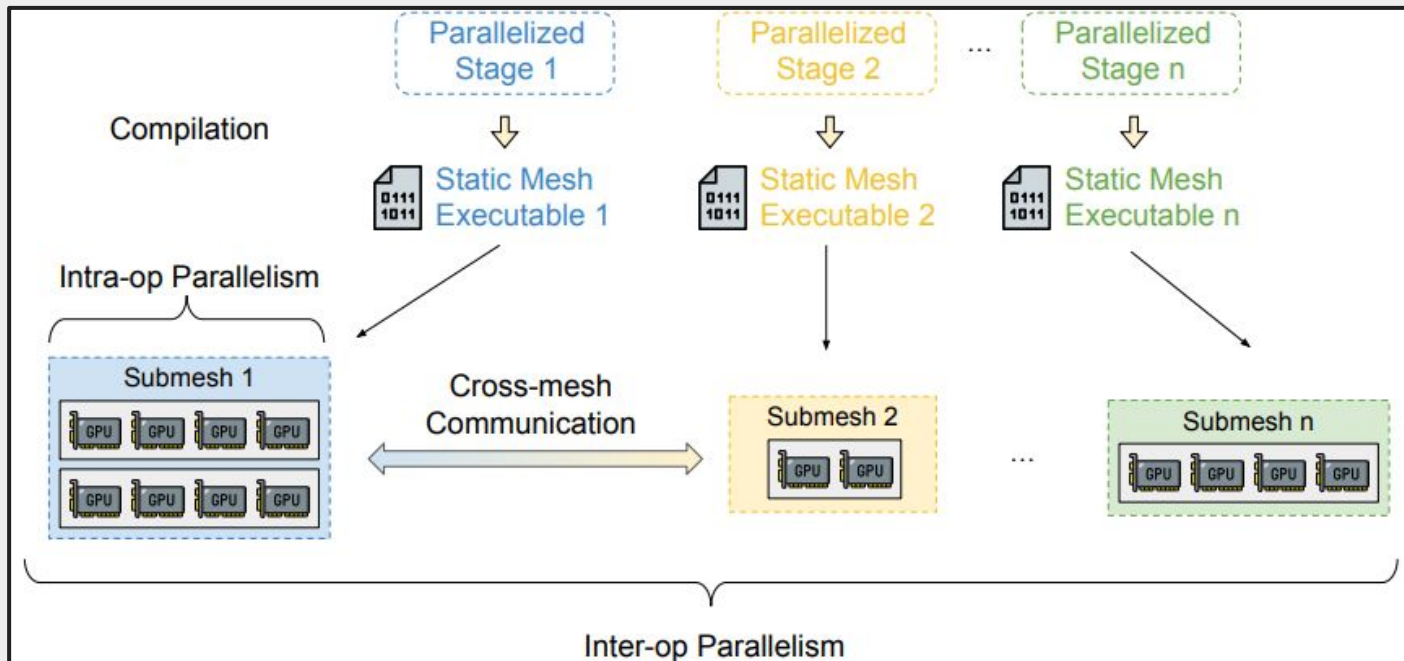
Parallel strategies of each operator

stage
input

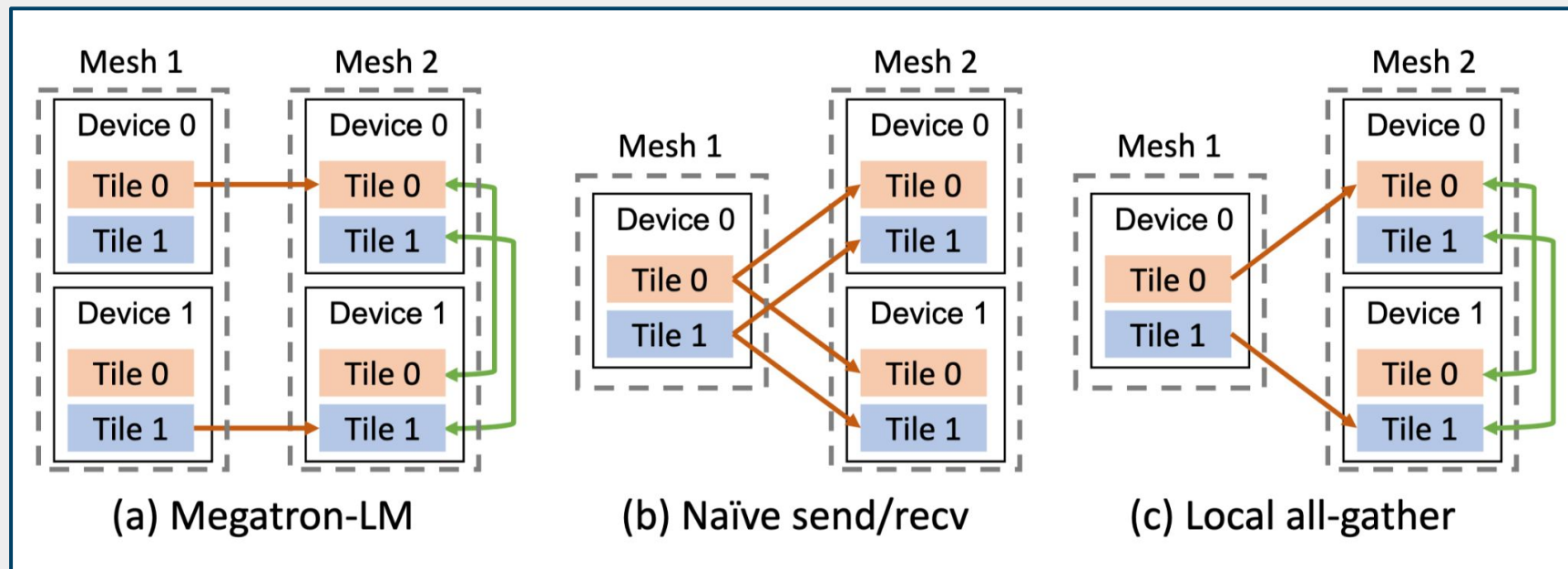
Minimize Com

#	Src Spec	Dst Spec	Communication Cost
1	RR	$S^0 S^1$	0
2	$S^0 R$	RR	$all-gather(M, 0)$
3	$S^0 S^1$	$S^0 R$	$all-gather(\frac{M}{n_0}, 1)$
4	$S^0 R$	RS^0	$all-to-all(\frac{M}{n_0}, 0)$
5	$S^0 S^1$	$S^0 R$	$all-to-all(\frac{M}{n_0 \cdot n_1}, 1)$

Runtime Orchestration



Runtime Orchestration





Evaluation

Models (size: B):

- GPT-3
- GShard MoE
- Wide-ResNet

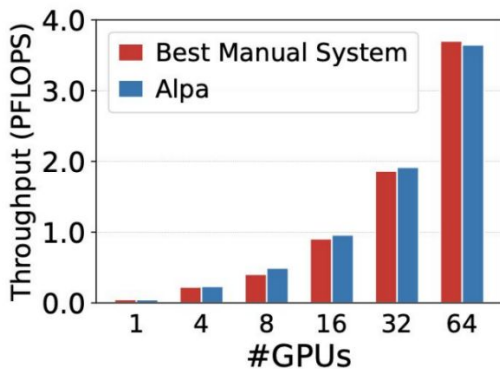
Specs:

- 8 nodes, 64 GPUs
- Each node: AWS EC2 p3.16xlarge with
 - GPU: 8 * NVIDIA V100 16GB
 - vCPUs: 64
 - Mem: 488GB



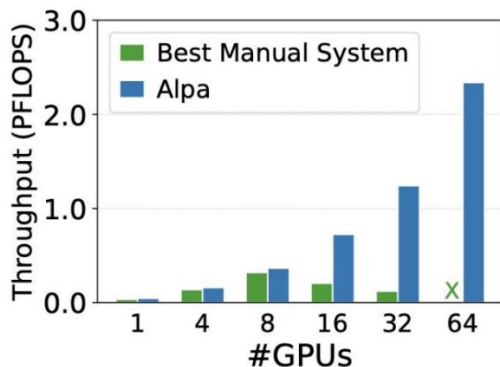
Evaluation

GPT (up to 39B)



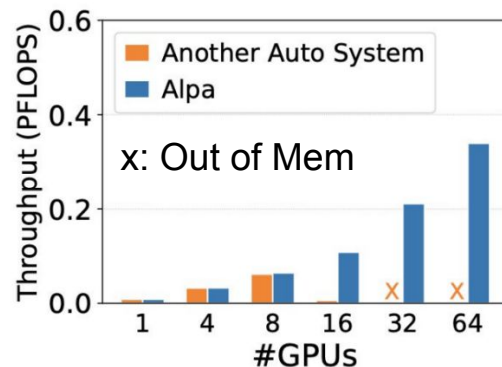
Match specialized manual systems.

GShard MoE (up to 70B)



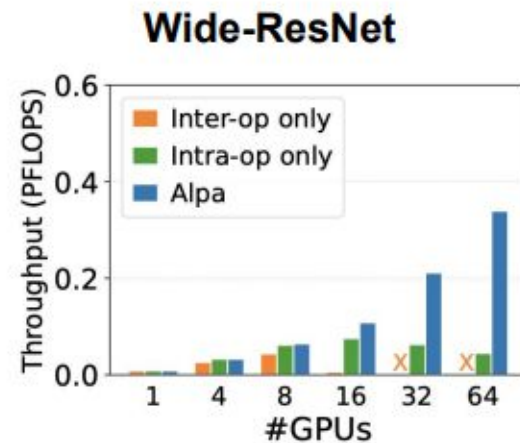
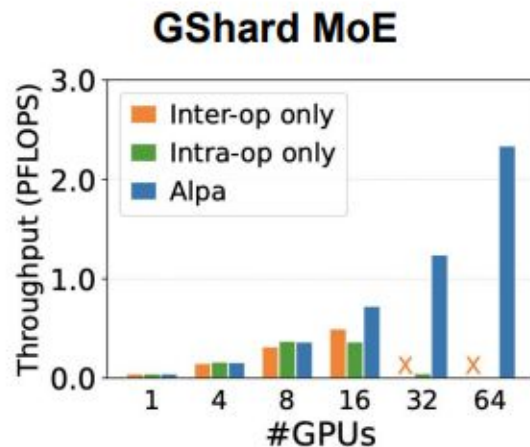
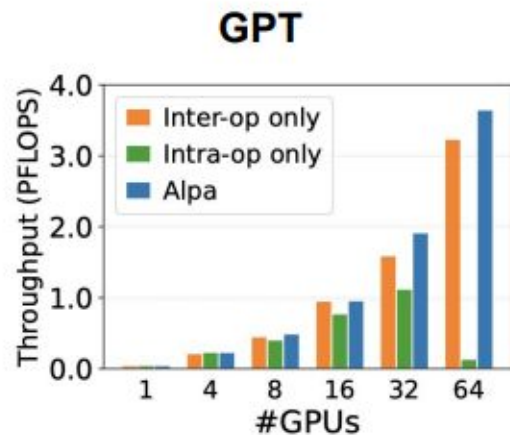
Outperform the manual baseline by up to 8x.

Wide-ResNet (up to 13B)



Generalize to models without manual plans.

Evaluation



Combining inter- and intra-operator parallelism scales to more devices.



Limitations & Future improvements

- Alpa can only deal with “static graph”
→ All tensor shapes **must** be known at compilation time.
- Inter-op pass pipelined with only static linear schedule
- Does not model communication cost between different stages.

PartIR: Composing SPMD Partitioning Strategies for Machine Learning (optional)

Why auto compilation is not enough?

- Unpredictability
- Long compilation time

automatic partitioning +
known and easy-to-apply
manual strategies

Table 5: Compilation time breakdown of GPT-39B.

Steps	Ours	w/o optimization
Compilation	1582.66 s	> 16hr
Profiling	804.48 s	> 24hr
Stage Construction DP	1.65 s	N/A
Other	4.47 s	N/A
Total	2393.26 s	> 40hr



What is PartIR?

- hardware-and-runtime agnostic NN partitioning system.
1. Expressive
 2. Decoupled
 3. Predictable

FSMoE: A Flexible and Scalable Training System for Sparse Mixture-of-Experts Models

FFN Layer -> MoE Layer

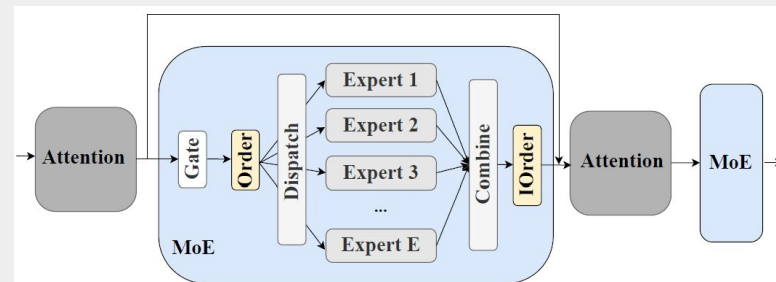
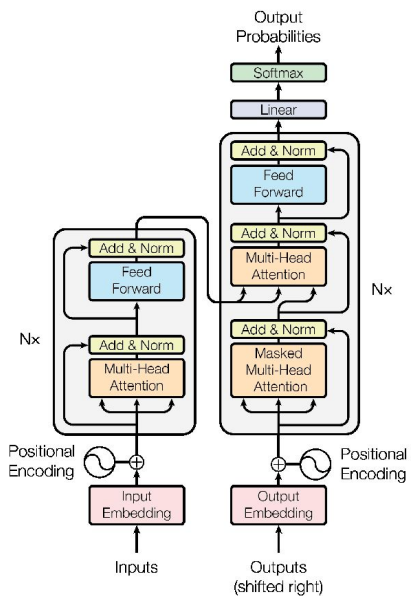
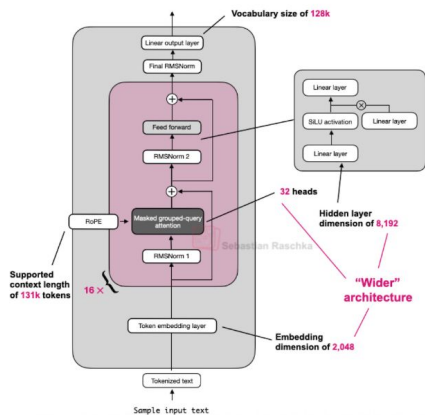


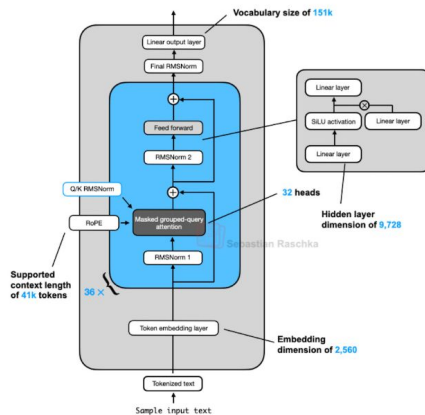
Figure 1. A typical MoE structure with E experts.



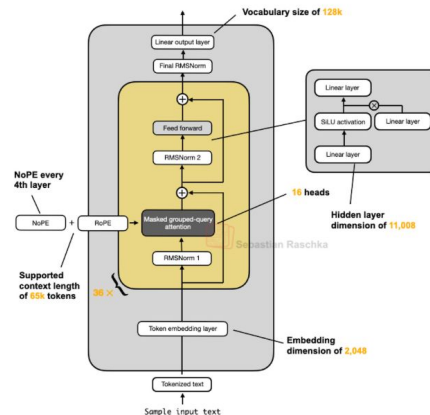
Llama 3.2 1B



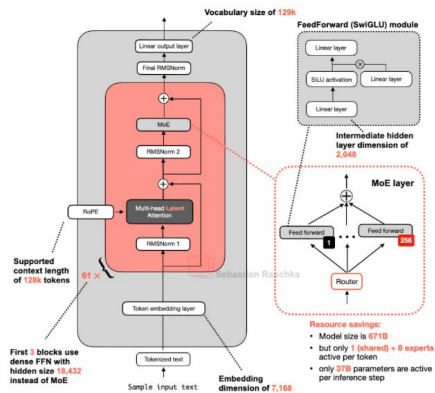
Qwen3 4B



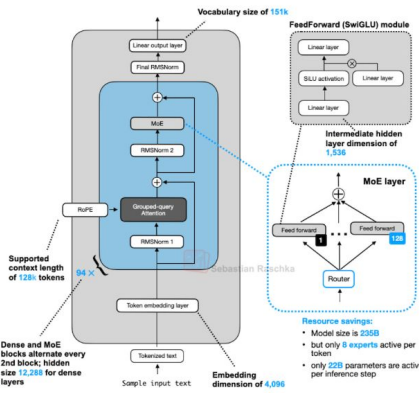
SmolLM3 3B



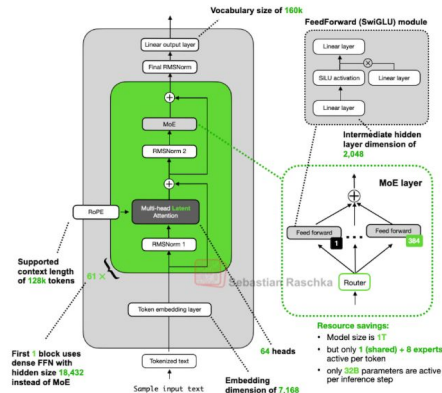
DeepSeek V3 (671B)



Qwen3 235B-A22B

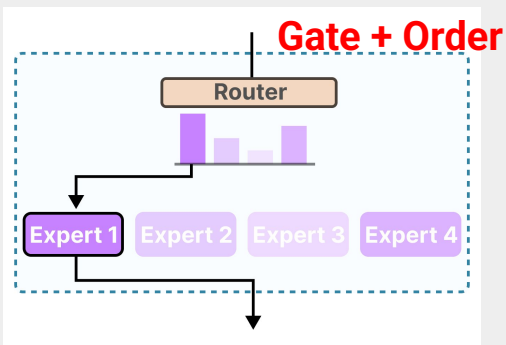


Kimi K2 (1 trillion)



Mixture of Experts (MoE)

Our Task: e.g. Sentence Boundary Detection



Layer 1	Expert 1	Expert 2	Expert 3	Expert 4
	Punctuation (, . : & - ? , etc.)	Verbs (said, read, miss, etc.)	Conjunctions (the, and, if, not, etc.)	Visual Descriptions (dark, outer, yellow, etc.)

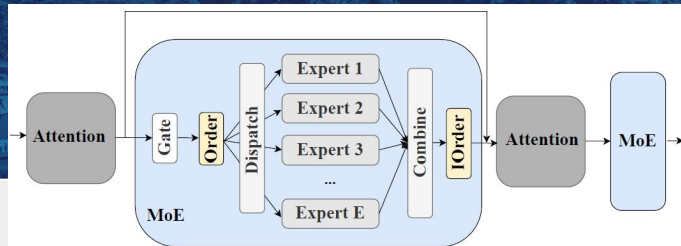
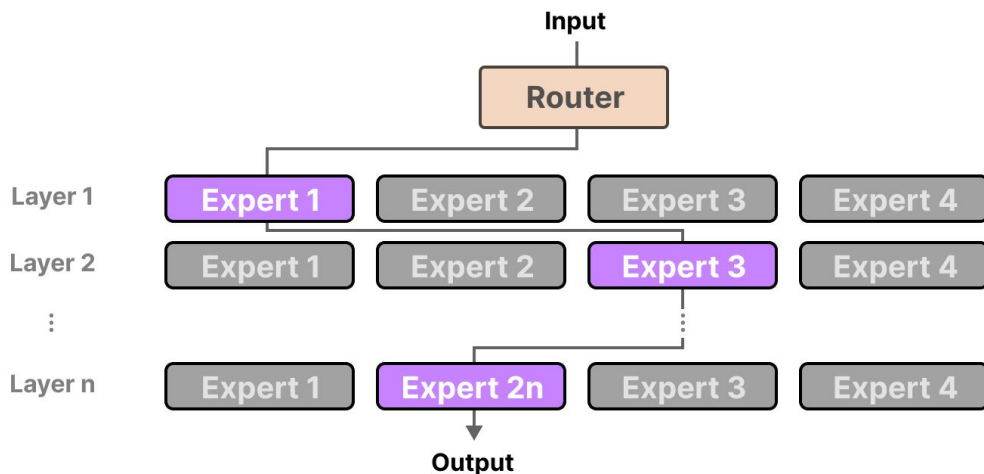


Figure 1. A typical MoE structure with E experts.



Paradigms of Parallelism (DP, MP, EP & ESP)

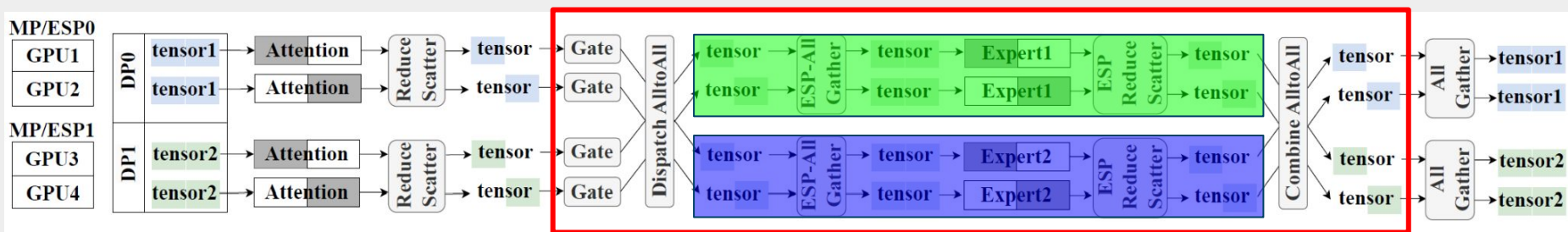


Figure 2. An example of $N_{DP} = N_{MP} = N_{EP} = N_{ESP} = 2$. The attention is partitioned into two parts across MP groups, and the two experts are distributed to the two EP groups (GPU1 and GPU3, as well as GPU2 and GPU4) in EP, and each expert is further partitioned into two shards across the ESP group. The blue and green rectangles indicate the data tensors.

Assumption:

- The group of MP and ESP matches the number of GPUs per node.
- Expert's model is considerably large, preventing it from fitting entirely on a single GPU.

- **Mixture-of-Experts Layer** (Red Box)
- **Expert Parallelism** (2 experts: Green & Blue)
- **Expert-Sharding Parallelism** (2 GPUs within each ESP group)
 - Green: Expert #1 (GPU #1 and #2)
 - Blue: Expert #2 (GPU #3 and #4)

Communication Overhead Dominates MoE Training Time

- **Communication overhead typically contributes over 50% to the overall training**
 - Necessity of optimizing communication performance

Table 2. Time performance (iteration time in millisecond) of each operation in a transformer layer of two real-world models, GPT2-XL [38] and Mixtral7B [20], with $B = 4$ and $L = 1024$ for two testbeds in Table 3. The numbers in the brackets represent each operation's portion of the forward and backward time.

Testbeds/Breakdown		Communication				Computation			
		AlltoAll	AllReduce	AllGather	ReduceScatter	Experts	Routing	Order	Attention
A	GPT2-Forward	6.9(31.16%)	0(0%)	4.6(20.83%)	5.4(24.46%)	3.1(14.04%)	0.1(0.45%)	0.3(1.36%)	1.7(7.7%)
	GPT2-Backward	6.9(21.27%)	5.26(16.26%)	4.6(14.22%)	5.4(16.7%)	6.1(18.86%)	0.1(0.31%)	0.4(1.24%)	3.6(11.13%)
	Mixtral-Forward	19.5(29.8%)	0(0%)	12.3(18.73%)	13.7(20.86%)	15.6(23.76%)	0.1(0.15%)	0.3(0.46%)	4.1(6.24%)
	Mixtral-Backward	19.6(17.45%)	26.45(23.59%)	12.3(10.97%)	13.7(12.22%)	31.8(28.36%)	0.1(0.09%)	0.5(0.45%)	7.7(6.87%)
B	GPT2-Forward	11.2(20.7%)	0(0.0%)	15.5(28.7%)	15.7(29.1%)	6.7(12.4%)	0.1(0.2%)	0.3(0.6%)	4.5(8.3%)
	GPT2-Backward	11.2(15.7%)	7.3(10.3%)	15.5(21.8%)	15.2(21.3%)	13(18.3%)	0.1(0.1%)	0.3(0.4%)	8.6(12.1%)
	Mixtral-Forward	28.3(15.9%)	0.0(0.0%)	39.6(22.3%)	40.8(23.0%)	58.5(33.0%)	0.1(0.1%)	0.7(0.4%)	9.5(5.4%)
	Mixtral-Backward	30.8(10.8%)	32.1(11.3%)	40.1(14.1%)	41.8(14.7%)	119.7(42.1%)	0.2(0.1%)	1.2(0.4%)	18.1(6.4%)

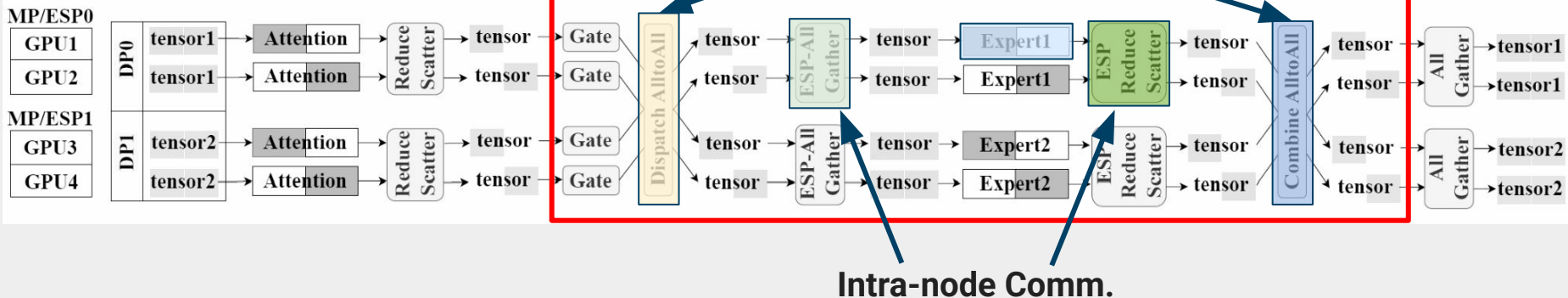
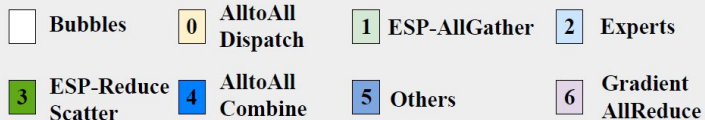
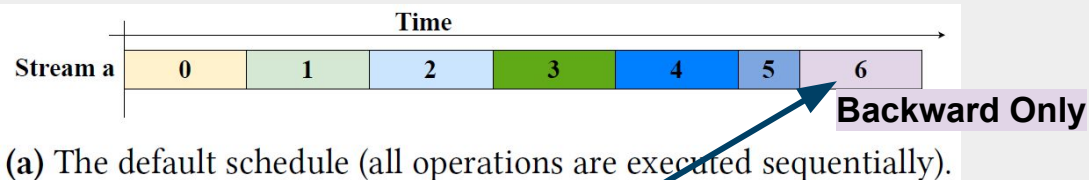
Overview of FSMoE

- **Modularized Operations**
 - Support all MoE layer components
 - Gating function, data layout, collective communication, expert computation, etc.
- **Adaptive Optimal Scheduling Algorithm**
 - Pipeline intra-node and inter-node communications with computation
 - Improve training efficiency in common DP + MP + EP + ESP scenarios
- **Adaptive Gradient Partitioning Method**
 - Overlap gradient aggregation with computation
 - Avoid contention between inter-node communications
- **Results**
 - Outperform Tutel and DeepSpeed-MoE by 1.18×~3.01× on GPT-2 and Mixtral with 32/48 GPU clusters

Motivations:

- **A Flexible MoE framework**
- **Optimizing Network Communication**
- **Optimizing Forward and Backward Separately**
- **Co-Design in Backward Propagation and Gradient Synchronization**

Scheduling





Optimized Scheduling of Tasks

Assumption: The group of MP and ESP is aligned with the number of GPUs in a node.

Tasks to Schedule:

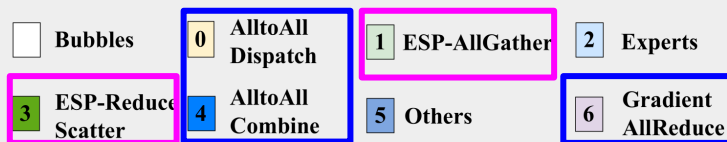
- Intra-node communications
 - ESP-AllGather
 - ESP-ReduceScatter
- Inter-node communications
 - AlltoAll Dispatch/Combine
 - Gradient-Allreduce
- Computing tasks
 - Expert computation
 - (Attention computation)

Hardware Communication:

- Intra-node communications
 - Faster, Shared Memory or NVLink
 - e.g., NVLink bandwidth: **900 GB/s**
- Inter-node communications
 - Slower, InfiniBand or Ethernet
 - e.g., 8×200 Gb/s NICs
→ **100 GB/s** peak bandwidth

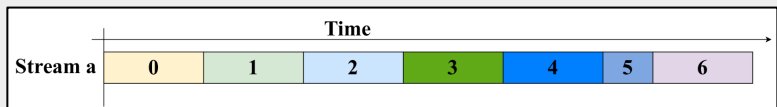
Goal: Design a new schedule to pipeline all time-consuming communications and computing tasks

4 Schedules

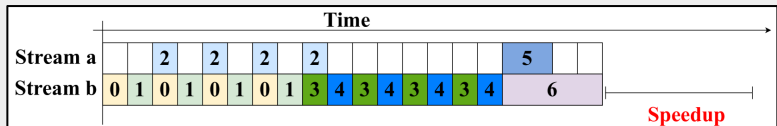


inter-node

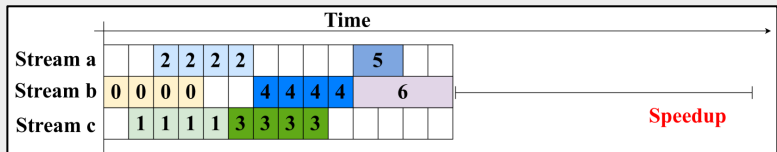
intra-node



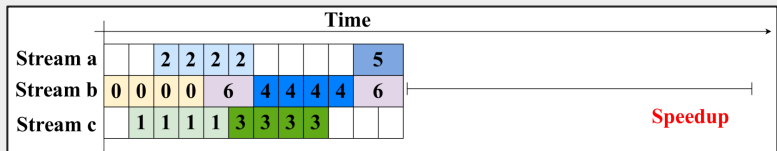
(a) **AlltoAll Dispatch**: sequentially



(b) **Tutel/PipeMoE w/ Gradient-AllReduce**:
AlltoAll & experts; AllReduce & non-MoE parts



(c) **FSMoE w/o partitioning gradient**:
Inter-node comm & Intra-node comm



(d) **FSMoE w/ partitioning gradient**:
Further overlap with AllReduce



Optimize Pipeline Degree r

- r : number of chunks per MoE layer pipeline
 - Linear Models # of communication message
- elapsed time startup time transmission time per byte

AlltoAll: $t_{a2a,r} = \alpha_{a2a} + \frac{n_{a2a}}{r} \cdot \beta_{a2a}$

AllGather: $t_{ag,r} = \alpha_{ag} + \frac{n_{ag}}{r} \cdot \beta_{ag}$

ReduceScatter: $t_{rs,r} = \alpha_{rs} + \frac{n_{rs}}{r} \cdot \beta_{rs}$

Expert: $t_{exp,r} = \alpha_{exp} + \frac{n_{exp}}{r} \cdot \beta_{exp}$

Tradeoff of r



7 Constraints

- Q1 (inter vs intra): $t_{a2a,r} > t_{ag,r}$
- Q2 (exp hides a2a): $r \cdot t_{exp,r} > 2(r - 1) \cdot t_{a2a,r}$
- Q3 (exp hides intra): $r \cdot t_{exp,r} > (r - 1) \cdot (t_{ag,r} + t_{rs,r})$
- Q4-7 (backpropagation): $t_{gar} > t_{ag,r} + t_{rs,r}$
 - Q4: $a2a > ag$, and $exp > a2a$
 - Q5: $a2a > ag$, $exp \approx a2a$
 - Q6: $a2a > ag$, $exp < intra\text{-}node$
 - Q7: $a2a \leq ag$, $exp > intra\text{-}node$

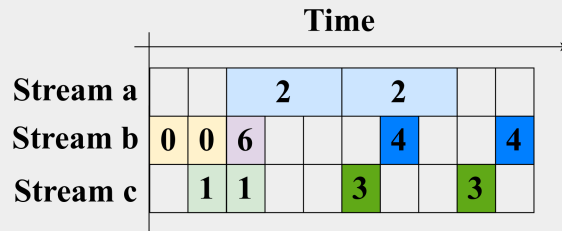
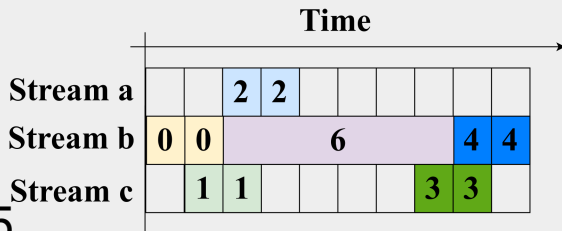
4 Cases



$r=2$

(1) Inter-node

$Q1 \wedge \neg Q2 \wedge Q4$
 $Q1 \wedge Q2 \wedge Q5$
 $\neg Q1 \wedge \neg Q3 \wedge Q5$
 $\neg Q1 \wedge Q3 \wedge Q7$

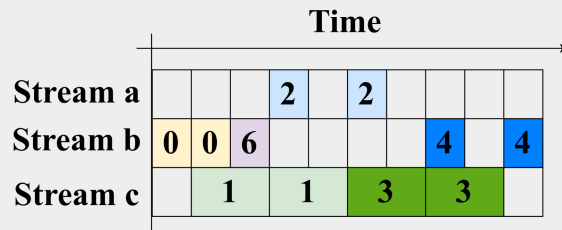
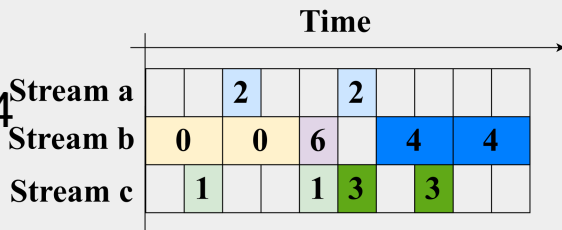


(2) Expert

$Q1 \wedge Q2 \wedge \neg Q5$
 $\neg Q1 \wedge Q3 \wedge \neg Q7$

(3) AlltoAll

$Q1 \wedge \neg Q2 \wedge \neg Q4$



(4) Intra-node

$Q1 \wedge \neg Q2 \wedge$
 $\neg Q4$

Find Optimal Pipeline Degree r

Algorithm 1 FindOptimalPipelineDegree

Input: $\alpha_{a2a}, \beta_{a2a}, n_{a2a}, \alpha_{ag}, \beta_{ag}, n_{ag}, \alpha_{rs}, \beta_{rs}, n_{rs}, \alpha_{exp}, \beta_{exp}, n_{exp}, t_{gar}$

Output: r and t^{moe}

```
1:  $r1, t1 = solve(f_1)$ 
2:  $r2, t2 = solve(f_2)$ 
3:  $r3, t3 = solve(f_3)$ 
4:  $r4, t4 = solve(f_4)$ 
5:  $candidate\_mins = [t1, t2, t3, t4]$ 
6:  $candidates = [r1, r2, r3, r4]$ 
7:  $r = candidates[\text{argmin}(candidate\_mins)]$ 
8:  $t^{moe} = \min(candidate\_mins)$ 
9: return  $r$  and  $t^{moe}$ .
```

▷ Solve with **SLSQP**

Sequential Least Squares Programming

- Iteratively find local optimal until global optimal
- Constrained minimization
- For each case, solve f w/ **SLSQP** to get candidate r
- Pick the r with smallest time cost



Scheduling for Backpropagation

- AllReduce and Inter-node MoE communications share the same bandwidth
- The optimal for forward is no longer optimal
- **Goal: maximize overlap of Gradient-AllReduce with MoE pipeline**

Scheduling for Backpropagation

- Performance Model: startup time

$$t_{ar}(n_{ar}) = \alpha_{ar} + n_{ar} \cdot \beta_{ar}$$

elapsed time

of communication message

transmission time per byte

- Inverse function: the max n in time t

$$g_{grad}^{inv}(t_{ar}) = (t_{ar} - \alpha_{ar}) / \beta_{ar}$$



Step 1: Calculate Partitioned Gradients

- Fill the bubbles with gradient from next layer
- Set $\tau_{ar} = 0$ (ignore backpropagation Gradient-AllReduce)
- Run Algo 1 to pick the best r across 4 scheduling cases
- Compute the time cost of overlappable parts

- Dense parts: measured before training

- Sparse MoE parts:

$$t_{olp,moe}(r) = \begin{cases} r \cdot t_{exp,r} + t_{ag,r} + t_{rs,r} - 2(r-1)t_{a2a,r}, & \text{Case 2} \\ t_{ag,r} + t_{rs,r}, & \text{Case 3} \\ r \cdot t_{ag,r} + r \cdot t_{rs,r} - 2(r-1)t_{a2a,r}, & \text{Case 4} \end{cases}$$

- Calculate the number of gradient; slice and assign them



Step 2: Optimize Partitioning

- **Assign remaining gradients after step 1**
- **Distribute remaining gradients across layers**
 - Minimize overall training time
- The optimization will be conducted only once before the training

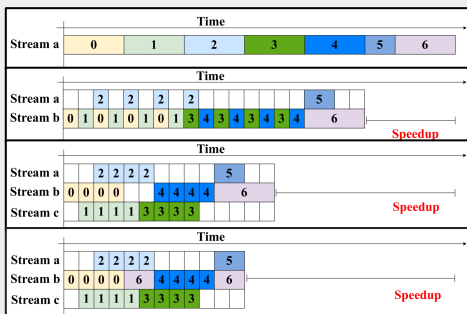


Evaluation

- Methods: **Tutel w/ PipeMoE** and **DeepSpeed-MoE**
- Testbeds:

Name	Testbed A	Testbed B
CPU	Dual Intel(R) Xeon(R) Platinum 8358 CPU @ 2.60GHz	Dual Intel(R) Xeon(R) Gold 6230 CPU @ 2.10GHz
GPU	8x Nvidia RTX A6000 @1.46GHz 48GB Mem 6 nodes, 48	4x Nvidia RTX 2080Ti @1.35GHz 11GB Memory 8 nodes, 32
Memory	512GB DDR4	512GB DDR4
NVlink	112.5GB/s (4x)	-
PCIe	4.0 (x16)	3.0 (x16)
Network	Mellanox MT28908 @ 200Gb/s	Mellanox MT27800 @ 100Gb/s

Evaluation: Configured Layers



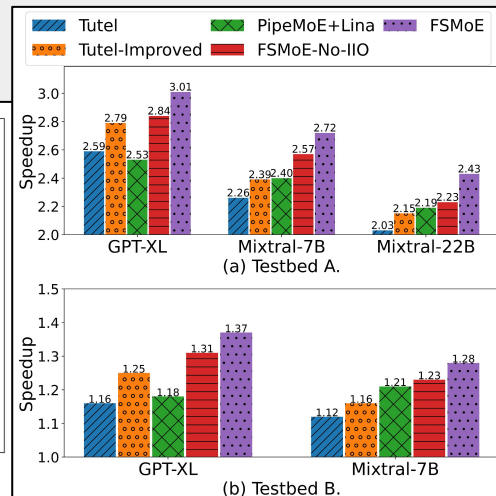
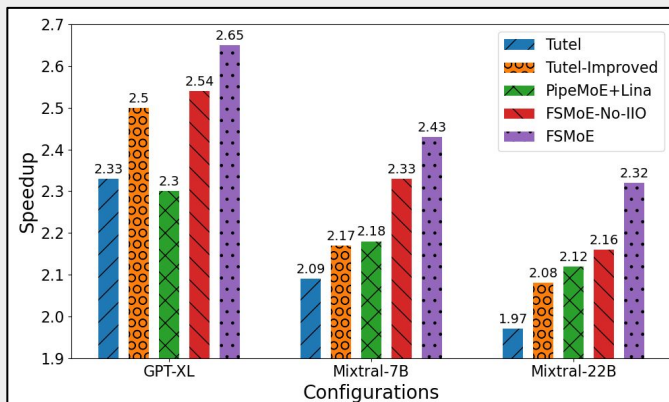
Schedule	Speedup	
	Testbed-A	Testbed-B
Tutel	1.00×	1.00×
Tutel-Improved	1.09×	1.08×
FSMoE-No-IIO	1.12×	1.16×
FSMoE	1.18×	1.22×

IIO overlap, and gradient partitioning brings speedup up to $\sim 1.22x$ over Tutel across 1458 configs.

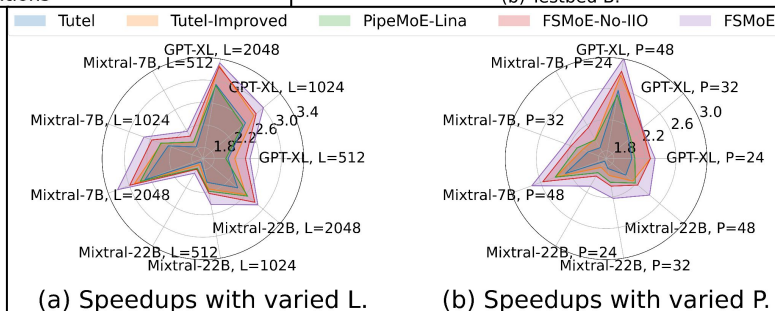
Evaluation: MoE Models

FSMoE is the best:

- MoE
- Configuration
- Gating



Gating	DeepSpeed-MoE	FSMoE
Gshard [22]	968.1 ± 1.4	707.7 ± 1.6(1.37×)
X-MoE [6]	1064.0 ± 1.5	746.9 ± 2.8(1.42×)
Sigmoid [23]	986.6 ± 1.4	721.0 ± 1.8(1.37×)
EC [52]	909.9 ± 1.8	685.5 ± 1.5(1.33×)





Limitations & Future improvements

- Assume time cost is linear model
- Cannot account for changes during training
 - Plan is made before training
- Performance results can be sensitive to environment setup
 - Hardware and CUDA versions
- Other routing/gating techniques?

FlexMoE: Scaling Large-scale Sparse Pre-trained Model Training via Dynamic Device Placement (optional)



Main Contributions

- **Fine-grained replicated expert parallelism** that selects specific heavy experts, duplicates them over multiple devices, and spreads the input tokens across the replicas
- **Dynamic expert management mechanism** to adaptively adjust the expert-to-device mapping and the assignment of tokens during training



With Large Expert Computation...

- FSMoE: arrange other tasks overlap with expert computation.
- FlexMoE: spread the long expert computation into multiple short ones.

Similarities and Differences



Similarities between the Two Papers

- Solved problems:
 - Imbalance between devices
 - Bottleneck from communication at some components
- Inter/Intra-node communication



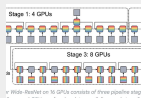
Differences between the Two Papers

- ALPA
 - General LLM
- FSMoE
 - Focus on imbalance between AlltoAll and others

Thank you!

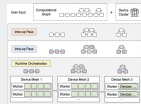


Image Sources



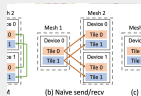
<https://research.google/blog/alpa-automated-model-parallel-deep-learning/>

Source: research.google



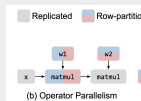
<https://www.usenix.org/conference/osdi22/presentation/zheng-lianmin>

Source: [Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning](https://www.usenix.org/conference/osdi22/presentation/zheng-lianmin)



<https://www.usenix.org/conference/osdi22/presentation/zheng-lianmin>

Source: [Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning](https://www.usenix.org/conference/osdi22/presentation/zheng-lianmin)

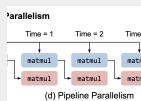


<https://www.usenix.org/conference/osdi22/presentation/zheng-lianmin>

Source: [Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning](https://www.usenix.org/conference/osdi22/presentation/zheng-lianmin)

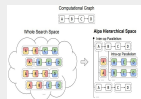


Image Sources



<https://www.usenix.org/conference/osdi22/presentation/zheng-lianmin>

Source: [Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning](#)



<https://www.usenix.org/conference/osdi22/presentation/zheng-lianmin>

Source: [Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning](#)

